CN2 - Generation of Decision Lists

Practical Assignment in Machine Learning

Martin Šulák

April 26, 2026

Contents

1 Introduction

The goal of this work is to implement the CN2 algorithm for generating decision lists and to verify its behaviour on categorical data. The result is a standalone Python application that loads a dataset from a CSV file, splits it into training and testing parts, trains a CN2 model, prints the learned rules, evaluates the classification, and stores textual as well as graphical outputs.

The topic belongs to the area of classification rule induction. A decision list is particularly suitable when, besides predictive performance, model interpretability is also important. The final model has the form of an ordered list of `IF - THEN` rules. During classification, the rules are tested in a fixed order and the first rule whose condition is satisfied is applied.

The work is focused on multiclass classification of student performance. The main dataset is the custom file `student_performance_clean.csv`, which contains three target classes of the attribute `vykon`: `nizky`, `stredny`, and `vysoky`. In addition to the clean version of the dataset, a noisy version `student_performance_noisy.csv` was also prepared in order to compare model behaviour on more ideal and less ideal data. The small `play_tennis.csv` dataset serves as an illustrative aid for explaining the CN2 principle.

The main objectives of the work are as follows:

- implement a custom CN2 classifier for categorical attributes,
- create a functional command-line application,
- verify the model on multiple datasets,
- evaluate accuracy, the number of rules, rule cost, and other characteristics,
- visualise the results and provide an interpretation of the experiments.

The report is divided into theoretical, design, implementation, and experimental parts. The final section summarises the achieved results and outlines possible extensions of the solution.

2 Theoretical Background

2.1 Decision Lists

A decision list is an ordered system of rules. Unlike an unordered rule set, the order of rules directly influences the classification result. When a new example is classified, rules are evaluated from top to bottom and the first rule whose condition is satisfied is used. If no explicit rule fires, a default rule is applied.

The general form of a decision list can be written as follows:

$$\begin{aligned} &\texttt{IF } D_1 \texttt{ THEN } C_1 \\ \texttt{ELSE IF } &D_2 \texttt{ THEN } C_2 \\ \texttt{ELSE IF } &D_3 \texttt{ THEN } C_3 \\ &\vdots \\ \texttt{ELSE } &C_{\text{default}} \end{aligned}$$

The main advantage of this model is good interpretability. A user can determine exactly which rule was applied and why the example was assigned to a particular class.

2.2 The CN2 Algorithm

The CN2 algorithm belongs to the family of algorithms that induce classification rules. It is a non-incremental algorithm that builds a decision list one rule at a time. In each iteration, it searches for the best complex, creates a new rule from it, removes all examples covered by that rule, and continues on the remaining examples.

CN2 uses a *general-to-specific* strategy. The search starts from the most general complex `TRUE`, and its specialisations are created by adding new selectors. Since the candidate space can grow very quickly, CN2 uses *beam search*, that is, a restriction on the number of best candidates stored in the set `STAR`.

The simplified learning procedure can be summarised as follows:

1. Generate all selectors of the form `attribute = value`.
2. Search for the best complex using beam search.
3. Assign the majority class of the covered examples to the complex.
4. Add the resulting rule to the decision list.
5. Remove the covered examples from the current training set.
6. Repeat until no further rule can be generated.

2.3 Selector, Complex, and Rule

A **selector** is the smallest condition, for example:

```
dochadzka = vysoka
```

A **complex** is a conjunction of several selectors, for example:

```
dochadzka = vysoka AND domace_ulohy = pravidelne
```

A **rule** is obtained by assigning a class to a complex, for example:

```
IF dochadzka = vysoka AND domace_ulohy = pravidelne THEN vykon = vysoky
```

2.4 Rule Quality Evaluation

In this implementation, the quality of a candidate complex is evaluated using the Laplace estimate. If a complex covers n examples and n_c of them belong to a given class, then the Laplace score is:

$$Q_{\text{Laplace}} = \frac{n_c + 1}{n + k},$$

where k is the number of classes.

This evaluation favours complexes that are both accurate and sufficiently general. It does not prefer extremely narrow rules that cover only a few examples.

2.5 Numerical Characteristics of a Rule

The following characteristics are used when interpreting rules:

- **rule cost** - the number of conditions in the complex,
- **consistency** - the proportion of correctly covered examples among all covered examples,
- **completeness** - the proportion of correctly covered examples of a given class among all examples of that class.

If a rule covers K examples and correctly covers K_r of them, then:

$$\text{consistency} = \frac{K_r}{K}$$

and if there are N_r examples of that class in the training set, then:

$$\text{completeness} = \frac{K_r}{N_r}.$$

These characteristics make it possible to evaluate not only the correctness of a rule, but also its practical usefulness.

3 Solution Design

The solution was designed as a standalone command-line Python application. The main reasons were straightforward execution, clear modularity, and the ability to test different datasets and parameter settings easily.

3.1 Project Architecture

The project is divided into several files according to responsibility:

- `main.py` - application entry point,
- `cn2.py` - implementation of the CN2 core,
- `rule.py` - representation of selectors, complexes, and rules,
- `data_utils.py` - dataset loading, validation, and splitting,
- `metrics.py` - metric computation and rule summarisation,
- `visualization.py` - generation of graphical outputs,
- `experiment_runner.py` - batch execution of experiments.

This structure improves readability and allows individual parts of the project to be modified without affecting the rest of the application.

3.2 Implementation Constraints

The implementation was intentionally designed for categorical attributes. Each attribute value is therefore interpreted as a text category. This choice was made for three reasons:

1. selectors of the type `attribute = value` can be implemented directly and simply,
2. the resulting rules are easy to read,
3. there is no need to solve the discretisation of continuous attributes.

Processing continuous attributes can be added later after discretisation or after extending the selector operator.

3.3 Application Inputs and Outputs

The application input is a CSV file in which the last column is treated as the target attribute. The application expects that:

- the dataset does not contain missing values,
- all attributes are categorical,
- the name of the target attribute is known at runtime,
- the target attribute is located in the last column.

The application outputs are:

- a text file with learned rules,
- a JSON file with metrics,
- CSV tables with the confusion matrix and rule summary,
- plots suitable for inclusion in the report.

3.4 Parameters Used

The most important parameters in the experiments were:

- `beam_width` - the number of candidates retained in the set `STAR`,
- `min_coverage` - the minimum number of covered examples required for a complex to be accepted,
- `max_rule_length` - the maximum number of conditions in one rule,
- `test_size` - the size of the test set during data splitting.

In the main run discussed in detail, the values were `beam_width = 5`, `min_coverage = 2`, and `max_rule_length = 4`.

4 Implementation

4.1 Classes `Selector`, `Complex`, and `Rule`

The file `rule.py` contains three basic data structures:

- `Selector` represents one condition of the form `attribute = value`,
- `Complex` represents a conjunction of selectors,
- `Rule` represents a classification rule in the form `IF complex THEN class`.

Each of these structures contains the method `matches()`, which determines whether a given example satisfies the relevant condition.

4.2 Model Training

The main model behaviour is implemented in the class `CN2Classifier`. When the method `fit()` is called, learning proceeds as follows:

1. a list of all selectors is created from the training set,
2. an empty decision list is initialised,
3. the best complex is searched on the current set,
4. the majority class of the covered examples is assigned to the complex,
5. the resulting rule is stored in the rule list,
6. the covered examples are removed,
7. the process is repeated until the stopping condition is reached.

If uncovered examples remain after the iterations finish, a default class is determined from them.

4.3 Searching for the Best Complex

A key part of the implementation is the method `_find_best_complex()`. This method performs beam search over the set `STAR`. First, an empty complex is inserted into `STAR`. Then the following steps are repeated:

1. specialise each complex in `STAR` by one new selector,
2. discard candidates that do not satisfy the minimum coverage,
3. compute the Laplace score for the remaining candidates,
4. sort the candidates by quality,
5. keep the best candidates according to `beam_width`.

In case of score ties, an auxiliary ordering is used: first larger majority-class coverage is preferred, then larger total coverage, and finally lower rule cost.

4.4 Metric Computation

After training, the following quantities are computed on the test set:

- **accuracy**,
- **confusion matrix**,
- **number of rules**,
- **average rule cost**,
- a table with details of the individual rules.

The file `rule_summary.csv` contains, for each rule, its textual form, Laplace score, coverage, cost, consistency, completeness, and the size of the training set on which the rule was learned.

4.5 Execution Example

The application is executed from the command line. The basic command for the main dataset is:

```
python main.py --dataset datasets/student_performance_clean.csv --target vykon
```

After execution, the rules, the main metrics, and the confusion matrix are printed to the console. At the same time, all textual and graphical outputs are stored in the `outputs` directory.

5 Datasets Used

5.1 Dataset `play_tennis.csv`

The dataset `play_tennis.csv` is a classic small illustrative dataset containing 14 examples. It is used only for explaining the principle of CN2 in a didactic way. It contains the attributes `outlook`, `temperature`, `humidity`, `wind`, and the target attribute `play`. The main advantage of this dataset is its small size and intuitive interpretation of rules.

5.2 Dataset `student_performance_clean.csv`

The main dataset contains 90 examples and was created specifically for this work. Each example represents a fictitious student and his or her behaviour with respect to several factors influencing performance. The input attributes are:

- `dochadzka`,
- `domace_ulohy`,
- `testy`,
- `aktivita`,
- `spanok`,

- priprava_pred_skusou,
- pouzivanie_mobilu,
- dochvilnost.

The target attribute is vykon with the classes nizky, stredny, and vysoky. The dataset is balanced, with each class containing 30 examples.

The dataset was created with logical consistency in mind. For example, combinations of high attendance, regular homework completion, and systematic exam preparation were more often associated with the class vysoky, while combinations of low attendance, missing preparation, and poor test results were more often associated with the class nizky. At the same time, some boundary cases were inserted so that the model would not be trivial.

5.3 Dataset student_performance_noisy.csv

The noisy version was created by modifying the clean dataset. For a subset of examples, one attribute value or the target class was changed. The purpose of this modification was to simulate less accurate data and test how the behaviour of the decision list changes in the presence of noise.

Comparing the clean and noisy datasets makes it possible to assess algorithm robustness and provides useful material for interpreting differences in the results.

6 Experiments

6.1 Experimental Setup

The experiments were designed to assess the influence of the parameter beam_width on model quality. The following values were tested:

```
beam_width = 3, 5, 7
```

For both main datasets, a stratified 80:20 split into training and test parts was used. This preserved approximately the same class proportions in both parts of the data.

The common experimental parameters were:

- min_coverage = 2,
- max_rule_length = 4,
- random_state = 42.

6.2 Metrics Tracked

The following quantities were recorded for each run:

- classification accuracy on the test set,
- number of learned rules,
- average rule cost,

- default class,
- confusion matrix.

This set of metrics makes it possible to assess not only predictive performance, but also model complexity and interpretability.

6.3 Main Analysed Run

This report discusses in greater detail the run on the dataset `student_performance_clean.csv` with parameters `beam_width = 5`, `min_coverage = 2`, and `max_rule_length = 4`. This run produced a decision list with 13 rules and achieved the best accuracy among the analysed configurations on the clean dataset.

7 Results

7.1 Main Application Run

In the main application run on the dataset `student_performance_clean.csv` with parameters `beam_width = 5`, `min_coverage = 2`, and `max_rule_length = 4`, the following results were obtained:

- training set size: 72 examples,
- test set size: 18 examples,
- number of learned rules: 13,
- average rule cost: 1.8462,
- default class: `vysoky`,
- accuracy: 0.9444.

The model therefore classified 17 out of 18 test examples correctly. From the perspective of interpretability, it is important that the average rule cost remained low, so most of the rules are short and easy to read.

7.2 Confusion Matrix of the Main Run

Table 1 shows the confusion matrix for the main run on the clean dataset.

Table 1: Confusion matrix for `student_performance_clean.csv`.

Actual class	`nizky`	`stredny`	`vysoky`
`nizky`	6	0	0
`stredny`	0	6	0
`vysoky`	0	1	5

The table shows that the classes `nizky` and `stredny` were recognised without error. The only mistake occurred for the class `vysoky`, where one example was classified as

stredny. This is therefore a relatively mild error between neighbouring classes rather than an extreme misclassification.

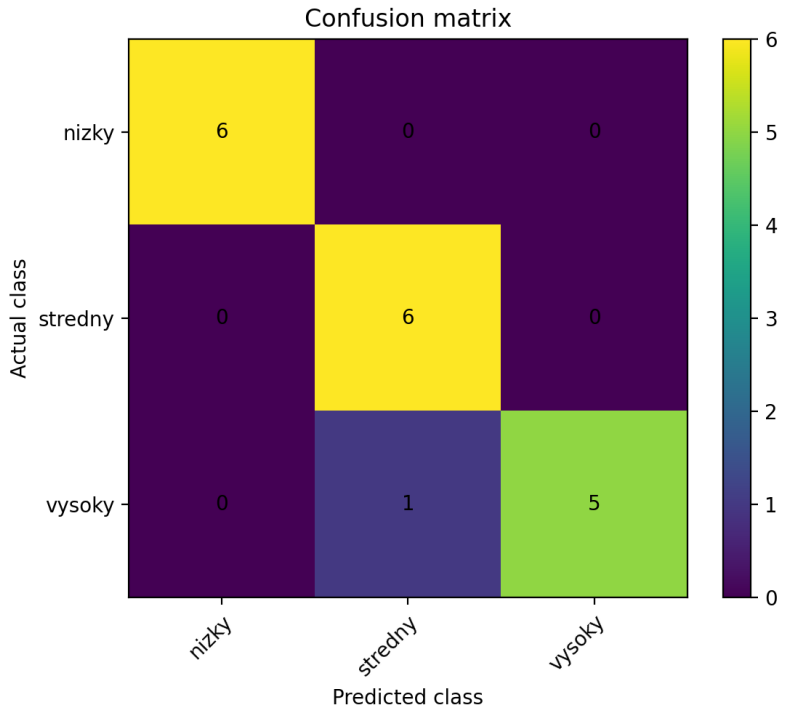

Figure 1: Graphical representation of the confusion matrix for the clean dataset.

7.3 Learned Rules of the Main Run

Table 2 presents the complete decision list learned in the main run. The last item in the list is the default rule with the class vysoky.

Table 2: Learned rules for the main run on the clean dataset.

ID	Rule	Cost
1	IF dochadzka = vysoka AND dochvilnost = dobra AND domace_ulohy = pravidelne THEN vysoky	3
2	IF dochadzka = nizka AND priprava_pred_skusou = ziadna THEN nizky	2
3	IF aktivita = stredna AND testy = priemerne THEN stredny	2
4	IF dochvilnost = zla AND domace_ulohy = zriedka THEN nizky	2
5	IF pouzivanie_mobilu = nizke AND priprava_pred_skusou = systematicka THEN vysoky	2
6	IF aktivita = vysoka AND testy = priemerne THEN stredny	2
7	IF domace_ulohy = obcas AND pouzivanie_mobilu = stredne THEN stredny	2
8	IF aktivita = vysoka THEN vysoky	1
9	IF spanok = nedostatocny AND testy = slabe THEN nizky	2

ID	Rule	Cost
10	`IF domace_ulohy = pravidelne AND pouzivanie_mobilu = vysoke THEN stredny`	2
11	`IF dochadzka = stredna AND dochvilnost = priemerna THEN stredny`	2
12	`IF testy = slabe THEN nizky`	1
13	`IF dochadzka = nizka THEN nizky`	1
D	`Default: vysoky`	0

These rules also appear logical from a domain point of view. The strongest positive indicators of high performance include high attendance, good punctuality, regular homework completion, and systematic preparation. In contrast, low performance is associated with low attendance, missing preparation, poor punctuality, or weak test results.

7.4 Rule Characteristics

The detailed table `rule_summary.csv` shows that the first rules had consistency equal to 1.0 and covered relatively large parts of the training set. This is expected because CN2 tries to find the highest-quality and most general rules in the early iterations. As the rule order increases, coverage typically decreases and the rules become more specific.

The strongest first rule was:

```
IF dochadzka = vysoka AND dochvilnost = dobra AND domace_ulohy =
                    pravidelne THEN vysoky
```

It covered 13 training examples, all of them correctly, and its Laplace score was 0.875. The final rules covered only small remnants of the training set and served mainly to complete the decision list.

Figure 2: Cost of individual rules on the clean dataset.

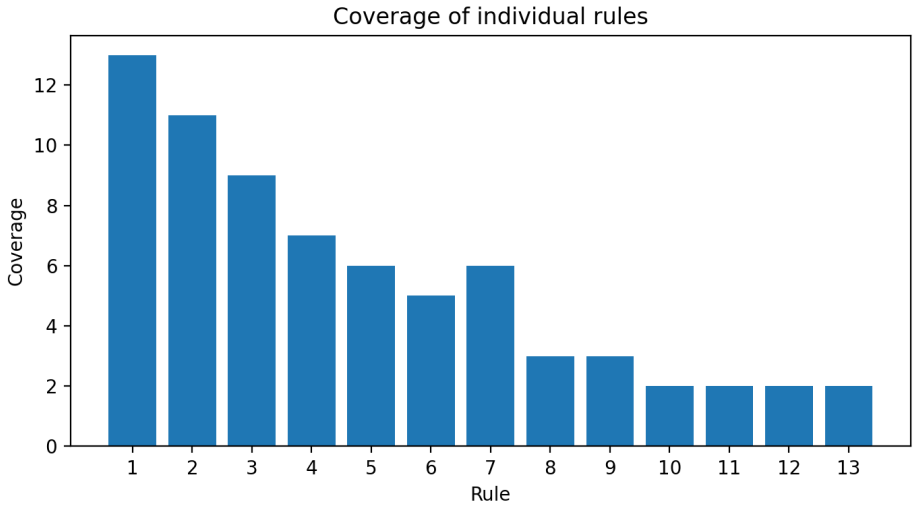

Figure 3: Coverage of individual rules on the clean dataset.

7.5 Comparison of Experiments

Table 3 summarises the experimental results for the clean and noisy datasets with different values of `beam_width`.

Table 3: Summary of experiments for the clean and noisy datasets.

Dataset	Beam width	Accuracy	Number of rules	Average cost
`student_performance_clean`	3	0.9444	13	1.8462
`student_performance_clean`	5	0.9444	13	1.8462
`student_performance_clean`	7	0.7778	13	1.9231
`student_performance_noisy`	3	0.4444	11	1.8182
`student_performance_noisy`	5	0.6111	11	1.8182
`student_performance_noisy`	7	0.6111	11	1.8182

Several interesting observations can be made from the results:

- On the clean dataset, the best results were achieved for `beam_width = 3` and `beam_width = 5`.
- Increasing `beam_width` to 7 did not improve the result; instead, it reduced accuracy to 0.7778.
- On the noisy dataset, accuracy was noticeably lower, which confirms the model's sensitivity to noise in the data.
- The number of rules remained stable within a given dataset, while accuracy changed.

This result shows that a larger candidate space does not automatically lead to a better model. In this task, a wider beam led to worse generalisation on the clean dataset.

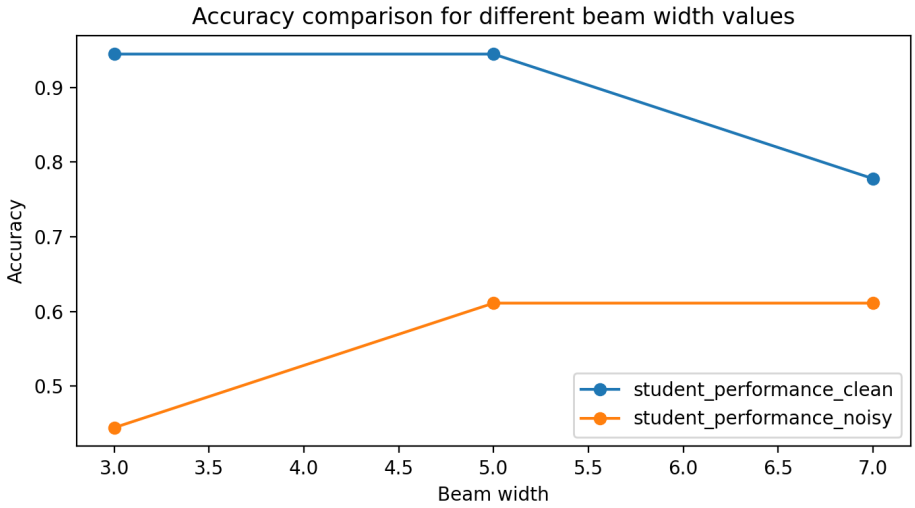

Figure 4: Accuracy comparison for different values of `beam_width`.

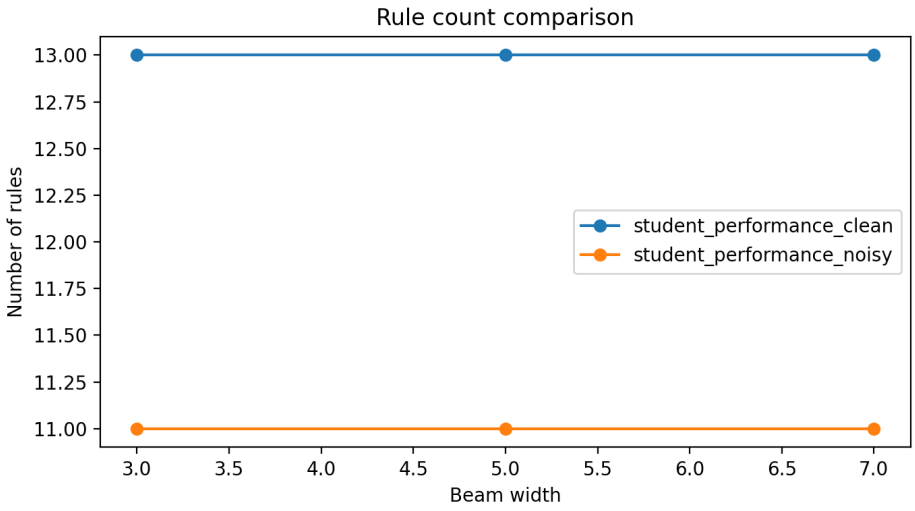

Figure 5: Rule count comparison for different values of `beam_width`.

7.6 Summary of Results

The experiments show that the implemented CN2 performs very well on clean categorical data. Besides high accuracy, it also retains very good interpretability because the final model is composed of short and meaningful rules. On noisy data, the expected drop in accuracy occurs, but the model remains usable and still produces a consistent decision list.

8 Conclusion

The result of this work is a complete implementation of the CN2 algorithm for generating decision lists on categorical data. The created application can load a dataset from a CSV file, verify its validity, split it into training and test parts, train a model, classify

new examples, compute metrics, and store the results as textual, tabular, and graphical outputs.

On the main dataset `student_performance_clean.csv`, the model achieved an accuracy of 0.9444 with parameters `beam_width = 5`, `min_coverage = 2`, and `max_rule_length = 4`. The decision list consisted of 13 rules and one default rule. The average rule cost was 1.8462, which confirms that the resulting model is not only accurate, but also interpretable.

Comparison with the dataset `student_performance_noisy.csv` showed the expected drop in accuracy. This confirms that the quality of the input data has a strong impact on the behaviour of a rule-based model. It also showed that increasing `beam_width` does not necessarily improve the result and may in some cases lead to worse generalisation.

The main contributions of the work are:

- a custom implementation of CN2 without using a prebuilt classifier,
- the use of multiple datasets including a noisy version,
- computation and interpretation of several metrics,
- generation of graphical outputs suitable for presentation,
- preservation of good interpretability of the resulting model.

Possible future extensions include:

- adding a significance test for candidate rules,
- handling continuous attributes after discretisation,
- comparing the solution with other rule-based or tree-based algorithms,
- extending the experiments with multiple runs using different random data splits.

Overall, it can be concluded that the goal of the work was achieved. The created solution is functional, interpretable, and provides a sufficient basis for further experimentation as well as for the project defence.